DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Loops and Money

CMSC 131 Bootcamp Block 6

Prepared by: Rene Andre Bedonia Jocsing

Published: September 09, 2026 **Deadline:** September 09, 2026

Prerequisites

One archive holds everything this block needs, including a project that already builds. You can start today without Git, without a GitHub account, and without last session's folder.

You need this archive. Unzip it somewhere permanent and work inside the folder it makes.

Session Objectives

- Explain why money isn't stored in floating point
- Represent pesos and centavos in a single whole-number register
- Compute a percentage without leaving integer arithmetic
- Carry a running value across loop iterations
- Print a fixed-point number so it reads as currency

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, `b1` through `b7`, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

The archive is a working project with today's files already in it. If you prefer to keep working in your own folder, copy these across instead.

File	What it's for
`b6_starter.asm`	Today's exercise, with the prompts written and the loop left to you
`interest.input`	A balance, a rate, and a term, fed to your program by `make check`
`interest.expected`	What a correct program prints for them
`b6_validation.py`	Prints the year-by-year figures for any case you give it
`b5_solution.asm`	Block 5's program, finished, if you missed that session

Ninety minutes, roughly fifteen on why money isn't floating point, twenty on percentages, fifteen on carrying a value across a loop, and forty on the exercise and its formatting.

Part 1: Why Not Floating Point

The obvious way to store ₱19.99 is as the number 19.99. Almost every real financial system refuses to do this. Why?

Floating point stores numbers in binary. One tenth in binary is a repeating fraction, the same way one third in decimal is 0.333 forever. It has to be cut off somewhere, so the stored value is very slightly wrong.

Add ten of those slightly-wrong tenths together and the error accumulates. In most languages, `0.1 + 0.2` doesn't equal `0.3`. In a bank, that difference is somebody's money. It compounds daily across millions of accounts.

The fix is to stop storing fractions. Keep the amount in **centavos**, as a whole number.

Amount	Stored as
₱1.00	100
₱19.99	1999
₱0.05	5
₱1,000,000.00	100000000

Nothing is approximate. Addition and subtraction are exact. Your register holds an ordinary integer, which is the only thing it was ever good at.

This is called **fixed point**. The decimal point isn't stored. Everyone agrees in advance that it sits two digits from the right.

The cost is range. A 32-bit signed register tops out near 2.1 billion, so in centavos that's about ₱21.4 million. That's fine for a class exercise. But it's why real systems use 64-bit values for money.

Part 2: Percentages Without Fractions

Interest of 5% means multiplying by 0.05, but you have no fractions. So use the trick from Block 2 and multiply before you divide.

Five percent of a balance is `balance * 5 / 100`:

```
        mov     eax, ebx            ; eax = balance in centavos
        mov     ecx, 5              ; 5 percent
        mul     ecx                 ; eax = balance * 5
        mov     edx, 0
        mov     ecx, 100
        div     ecx                 ; eax = balance * 5 / 100
```

For a rate with a decimal, like 3.75%, scale it. 3.75% is `375 / 10000`:

```
        mov     ecx, 375
        mul     ecx
        mov     edx, 0
        mov     ecx, 10000
        div     ecx
```

Three things go wrong here often. Dividing first, as in `balance / 100 * 5`, truncates the balance to whole pesos before applying the rate, losing up to 99 centavos every single iteration. Forgetting `mov edx,`

`0` before `div` is worse than usual here, since the `mul` you just did wrote into `edx`. If the product was small, `edx` is zero and you get away with it. If it was large, `edx` holds the high half and your division is nonsense. Clear it every time. And `div` discards the remainder whether you meant to lose it or not, so if you want a bank's rounding rather than plain truncation, that remainder is sitting in `edx`.

Part 3: Carrying a Value Across Iterations

Interest compounds, so each year's balance depends on the last. That means one register has to survive the whole loop while others get overwritten by `mul` and `div`.

```
        mov     esi, 100000       ; balance, in centavos: P1000.00
        mov     ecx, 5            ; five years

year_loop:
        ; interest = balance * 5 / 100
        mov     eax, esi
        mov     ebx, 5
        mul     ebx
        mov     edx, 0
        mov     ebx, 100
        div     ebx

        add     esi, eax          ; balance = balance + interest

        dec     ecx
        jnz     year_loop
```

`esi` holds the balance and nothing else touches it. `eax`, `ebx`, and `edx` are scratch, freely destroyed each pass. `ecx` counts down, using the cheaper counting-down loop from Block 5.

`mul` and `div` insist on `eax` and `edx`. `ecx` is the natural counter. That leaves `ebx`, `esi`, and `edi` free, so the value that must survive goes in one of those.

Choosing this at the start is much easier than discovering halfway through that your balance keeps getting wiped by a multiply.

Part 4: Printing It Back as Currency

The balance is 105000 and you want `1050.00`. Split it with one division:

```
        mov     eax, esi
        mov     edx, 0
        mov     ebx, 100
        div     ebx               ; eax = pesos, edx = centavos

        mov     ecx, edx          ; save centavos, print_int wants eax
        call    print_int         ; pesos

        mov     eax, dot
        call    print_string      ; the "."

        mov     eax, ecx
        call    print_int         ; centavos
```

That prints `1050.0` when the centavos are `0`, and `1050.5` when they're `5`, neither of which is right. Currency wants two digits.

Five centavos should print as `.05`*, not* `.5`*. What test and what extra output fixes that?*

You have both halves of the answer already. Block 5 gave you `cmp` and a conditional jump for the test. Part 4 of that block gave you `print_char`, which prints one character:

```
        mov     eax, '0'
        call    print_char
```

Work out for yourself which comparison goes around it. `print_int` will never do this for you. It has no idea it's printing the right-hand half of anything.

Where to Put All This

This program has more values to keep alive at once than the last few did. The balance, the rate, the term, the year you're on, the starting balance, and a divisor of 100 all matter at the same time. `mul` and `div` claim `eax` and `edx` regardless. Count them against the six registers you have and you come up short.

So the starter keeps two of them in memory instead, declared in `.data` and read back with brackets:

```
rate            dd  0
years           dd  0

        mov     [rate], eax             ; store into it
        cmp     ecx, [years]            ; read out of it
```

Take that on trust today. Block 7 is entirely about what those brackets mean and why a bare label is something different. What matters now is the reason you needed them, which is that six registers ran out. Running out is the normal condition.

Part 5: Exercise

Start from `b6_starter.asm`. It asks for the three inputs already, but leaves you the loop and the currency formatting.

```
cp b6_starter.asm interest.asm
```

Write a program that computes compound interest annually.

1. Prompt for a starting balance in pesos and centavos, read as a whole number of centavos.
2. Prompt for an annual interest rate as a whole percentage.
3. Prompt for the number of years.
4. For each year, compute the interest, add it to the balance, and print the year number and the new balance formatted as currency.
5. After the loop, print the total interest earned.

Some rules. Whole-number arithmetic only. No floating point and no FPU instructions. Centavos must always print as two digits.

Expected output:

```
Starting balance in centavos: 100000
Annual rate as a percent: 5
Number of years: 3
Year 1: 1050.00
Year 2: 1102.50
Year 3: 1157.62
Total interest earned: 157.62
```

Trace year 3 by hand before you trust the program. The balance entering it is 110250 centavos. Interest is 110250 × 5 = 551250, divided by 100 is 5512, and the remainder of 50 is discarded. So the balance becomes 115762, which prints as `1157.62`.

That discarded 50 is half a centavo the bank kept. Where in your code did the decision to keep it get made, and what would rounding instead look like?

Run `python b6_validation.py` to see those three years laid out with the discarded remainder in its own column, and pass it your own balance, rate, and term to check any other case:

```
python b6_validation.py 250000 7 5
```

On Linux that's `python3`, as it was in Block 2.

Checking it

```
make PROG=interest check
```

`interest.input` holds the three answers from the sample run above, one per line, in the order the program asks for them.

Testing Checklist

Core Functionality

- A balance of 100000 at 5% for 3 years gives the three lines above
- Centavos below 10 print with a leading zero, so 5 shows as `.05`
- A rate of 0 leaves the balance unchanged across every year
- A term of 0 years prints no year lines and zero total interest
- Total interest equals the final balance minus the starting balance
- Large balances near ₱20,000,000 don't produce a wrong answer through overflow
- `make PROG=interest check` prints `OK: interest matches interest.expected`

Common Pitfalls

- Storing the balance in pesos rather than centavos, losing all precision
- Dividing before multiplying when applying the rate
- Leaving `edx` dirty after a `mul` and before the following `div`
- Keeping the balance in a register that `mul` overwrites
- Printing centavos with `print_int` alone, so `5` shows as `.5`
- Recomputing interest from the original balance every year, which is simple interest, not compound

Architecture Review

What We Built

- A money representation with no approximation in it
- Percentage arithmetic that stays in whole numbers
- A loop carrying state across iterations without losing it to scratch instructions
- Currency formatting from a single division

Key Takeaways

1. **Money is never floating point.** Store the smallest unit as a whole number.
2. **Fixed point is a shared agreement**, not a stored decimal point.
3. **Multiply first, divide last**, and clear `edx` between them every time.

4. **Decide early which register survives the loop.** `mul` and `div` will take the others.
5. **The discarded remainder is a decision.** Truncating is a choice, and so is rounding.

Next Session Preview

- Where your data lives, and the difference between `.data` and `.bss`
- Labels as addresses, and the brackets that dereference them
- Addressing modes, and computing an address instead of naming one
- A first look at shifting and masking bits, which Laboratory Activity 1 is built on